

Design and Analysis of Algorithms

Recursion Tree

National University of Computer and Emerging Sciences,
Islamabad

Methods to solve recurrence

- Iteration method
- Recursion tree method
- Master Theorem

Divide and Conquer

The divide-and-conquer paradigm

What is a paradigm? One that serves as a pattern or model.

- **Divide** the problem into a number of subproblems
- **Conquer** the subproblems by solving them recursively. If small enough, just solve directly without recursion
- **Combine** the solutions of the subproblems into the solution for the original problem

Analyzing Divide-and-Conquer Algorithms

Let $T(n)$ be the running time on a problem of size n .

If problem is small enough, then solution takes constant time (this is a boundary condition, which will be assumed for all analyses)

It takes time to divide the problem into sub-problems at the beginning. Denote this work by $D(n)$.

Analyzing Divide-and-Conquer Algorithms

It takes time to combine the solutions at the end. Denote this by $C(n)$.

If we divide the problem into ‘a’ problems, each of which is ‘1/b’ times the original size (n), and since the time to solve a problem with input size ‘n/b’ is $T(n/b)$, and this is done ‘a’ times, the total time is:

$$T(n) = \Theta(1), n \text{ small (or } n \leq c \text{)}$$

$$T(n) = aT(n/b) + D(n) + C(n)$$

Example

Function(int number)

if number<=1

then return;

else

Function(number/2)

Function(number/2)

for(i=1 to number)

Display i;

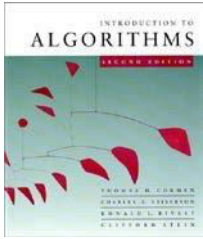
It follows that

$$T(n) = 2 T(n/2) + \textcircled{cn}$$

number of sub-problems

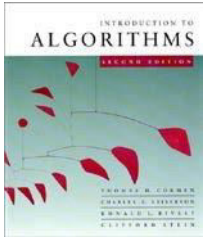
Sub-problem size

work done dividing and combining



Recursion tree

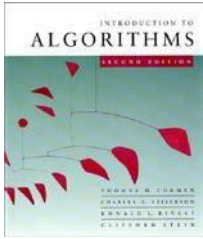
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Recursion tree

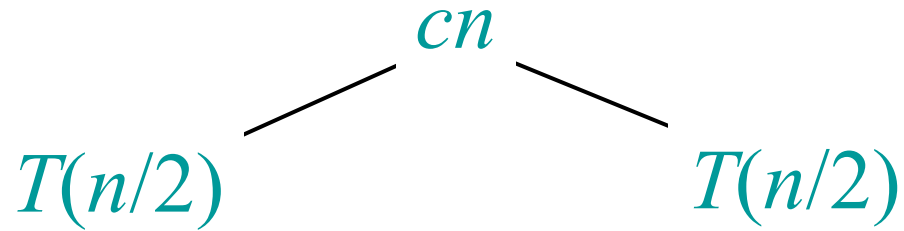
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

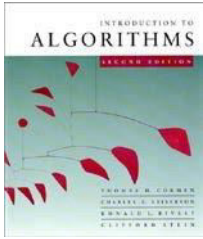
$$T(n)$$



Recursion tree

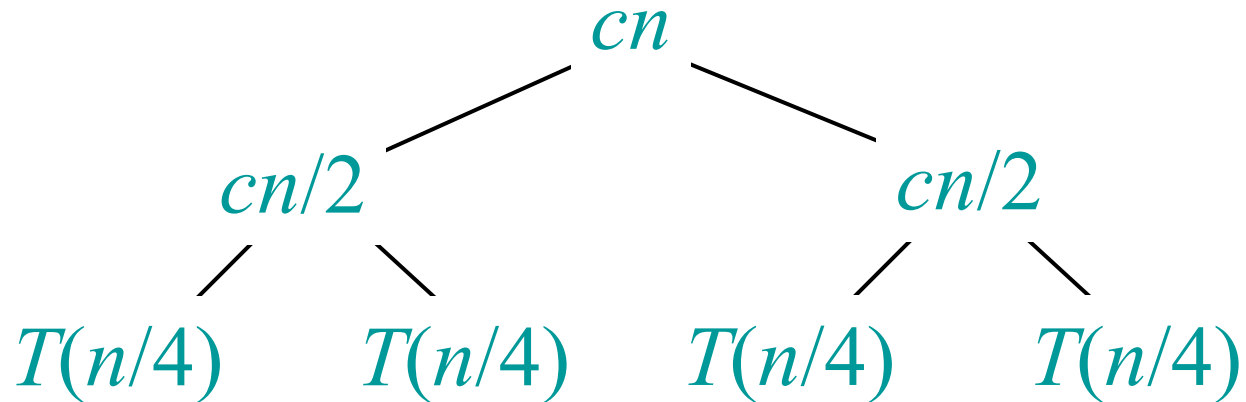
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

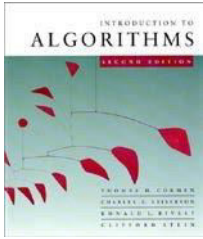




Recursion tree

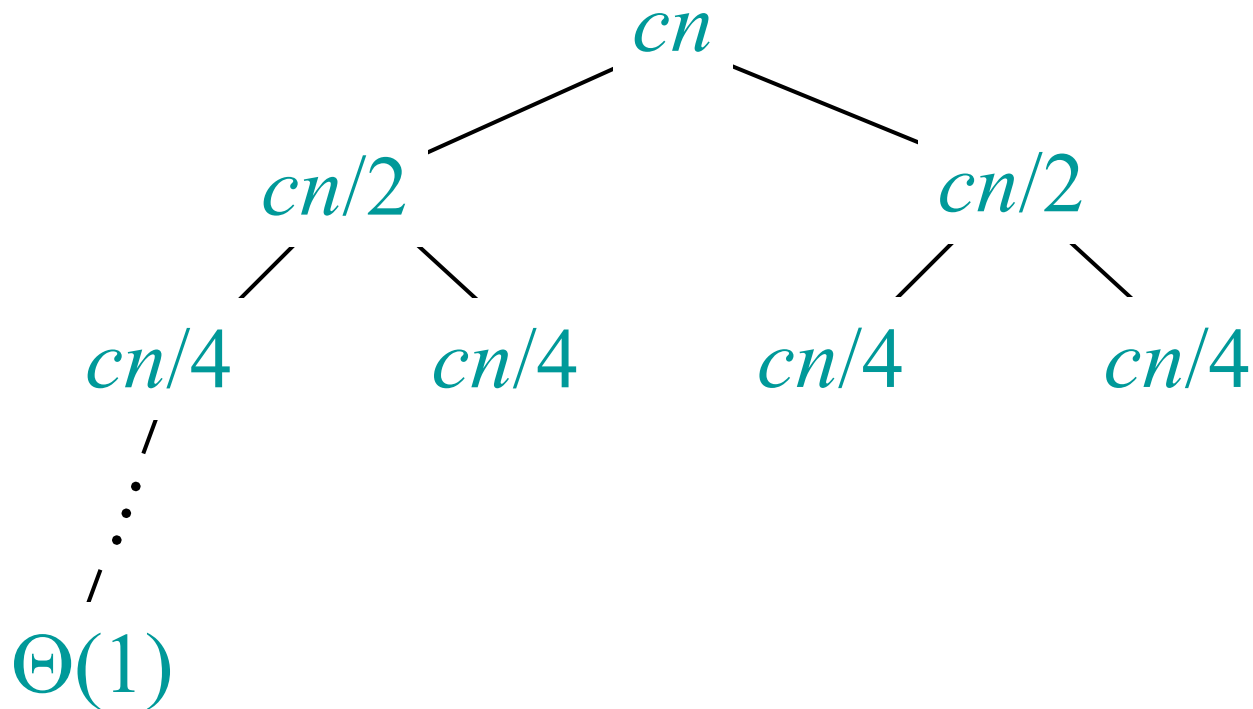
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

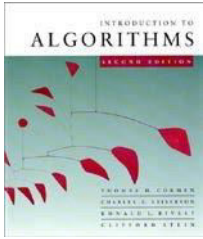




Recursion tree

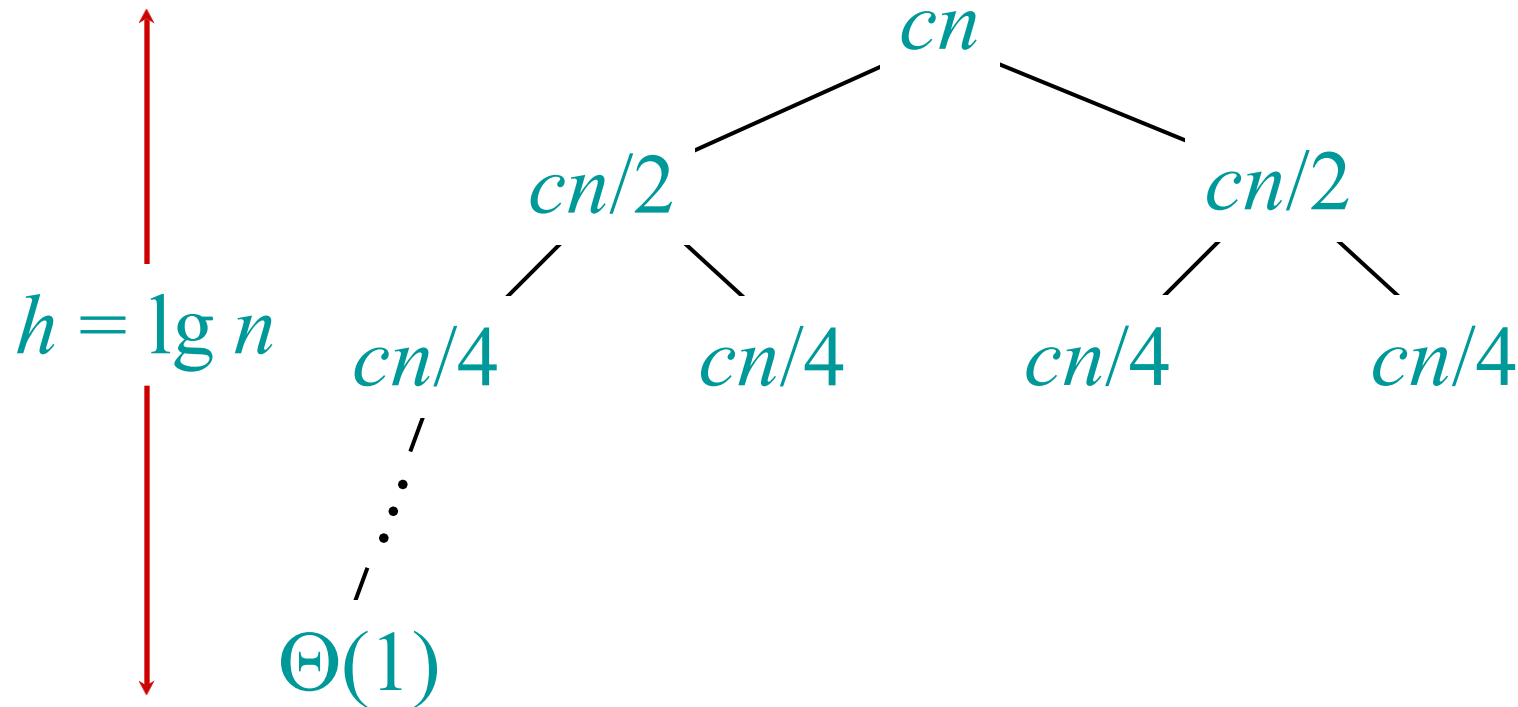
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

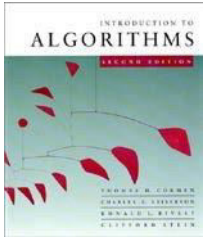




Recursion tree

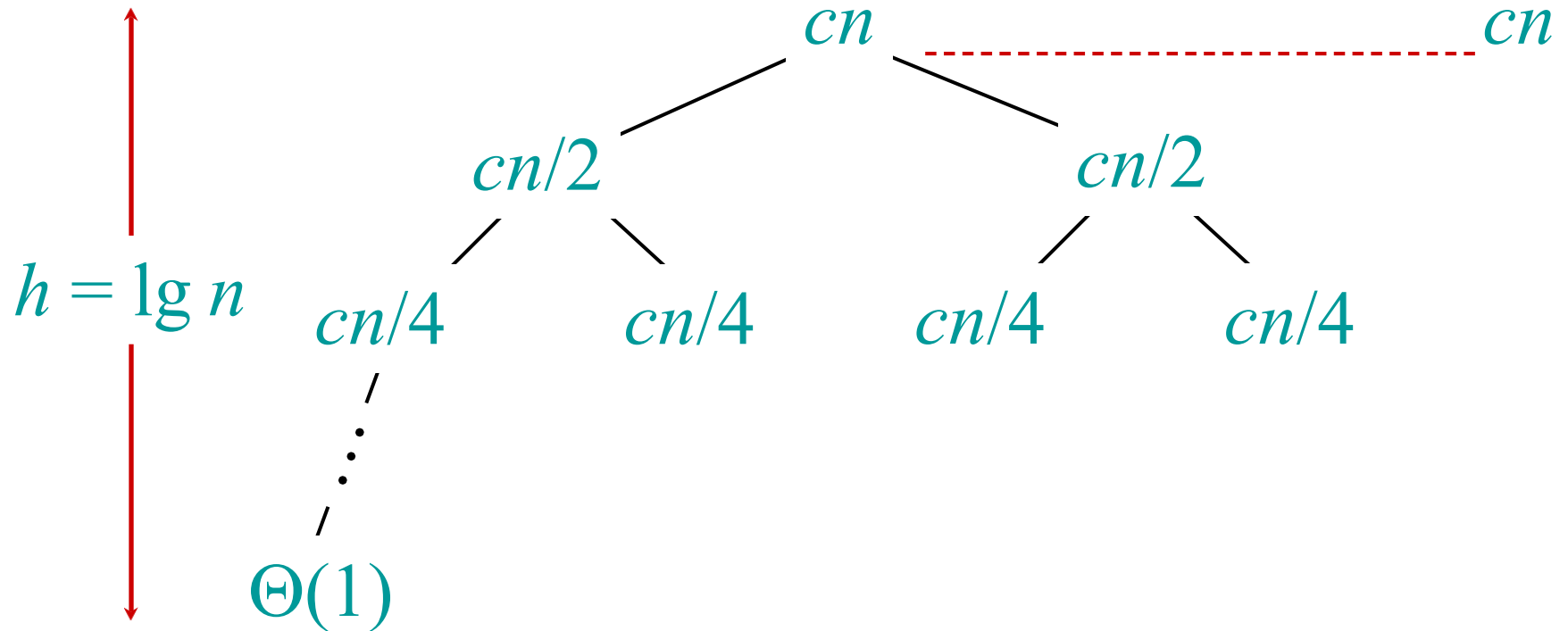
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

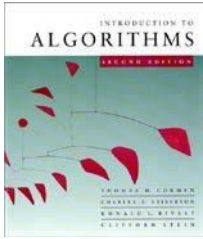




Recursion tree

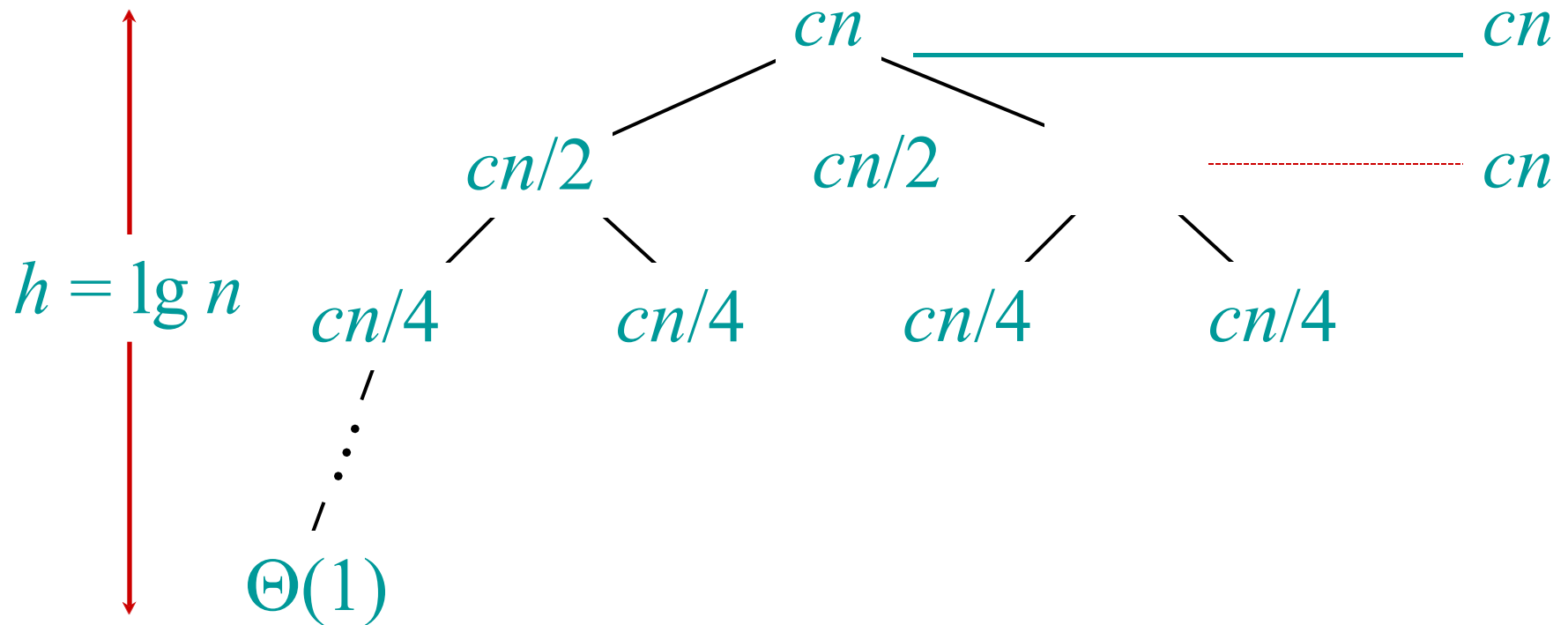
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

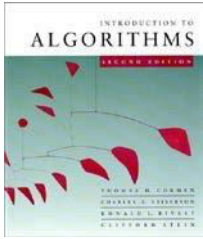




Recursion tree

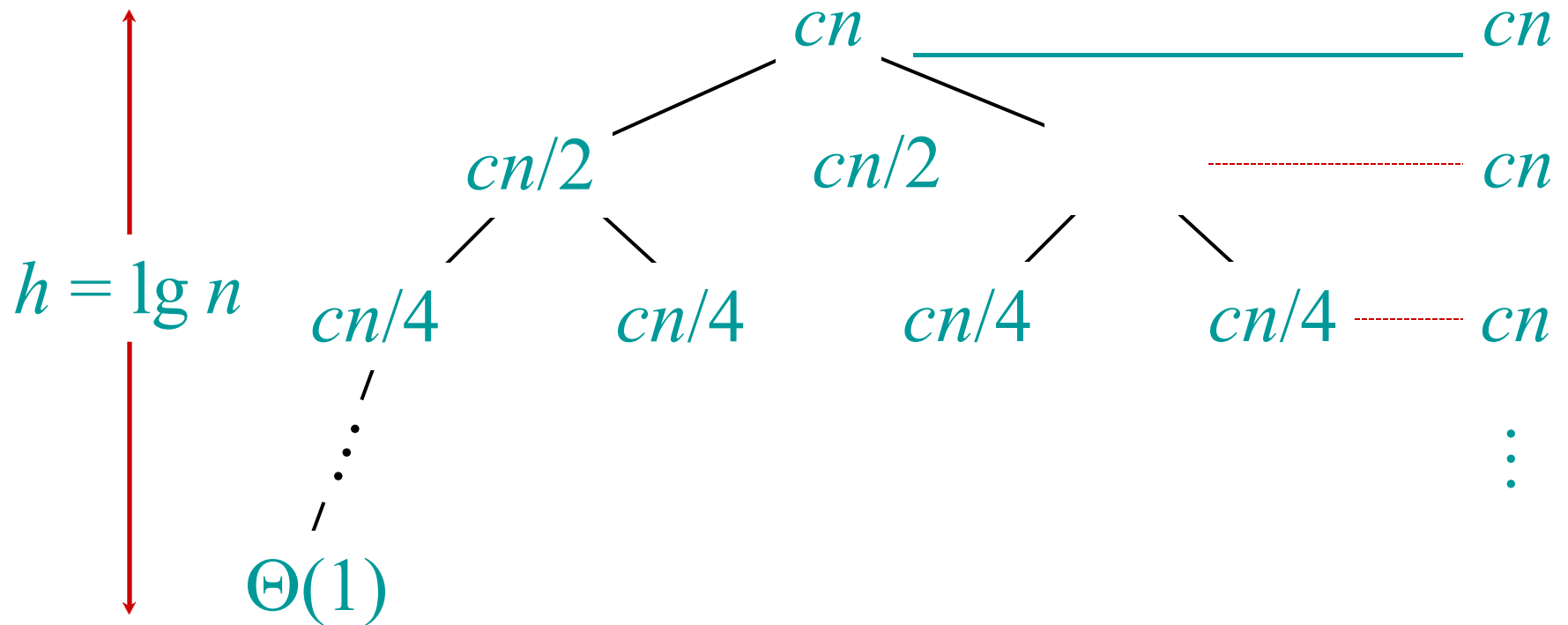
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

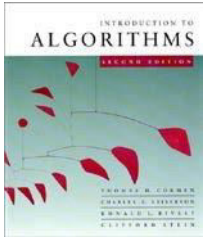




Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

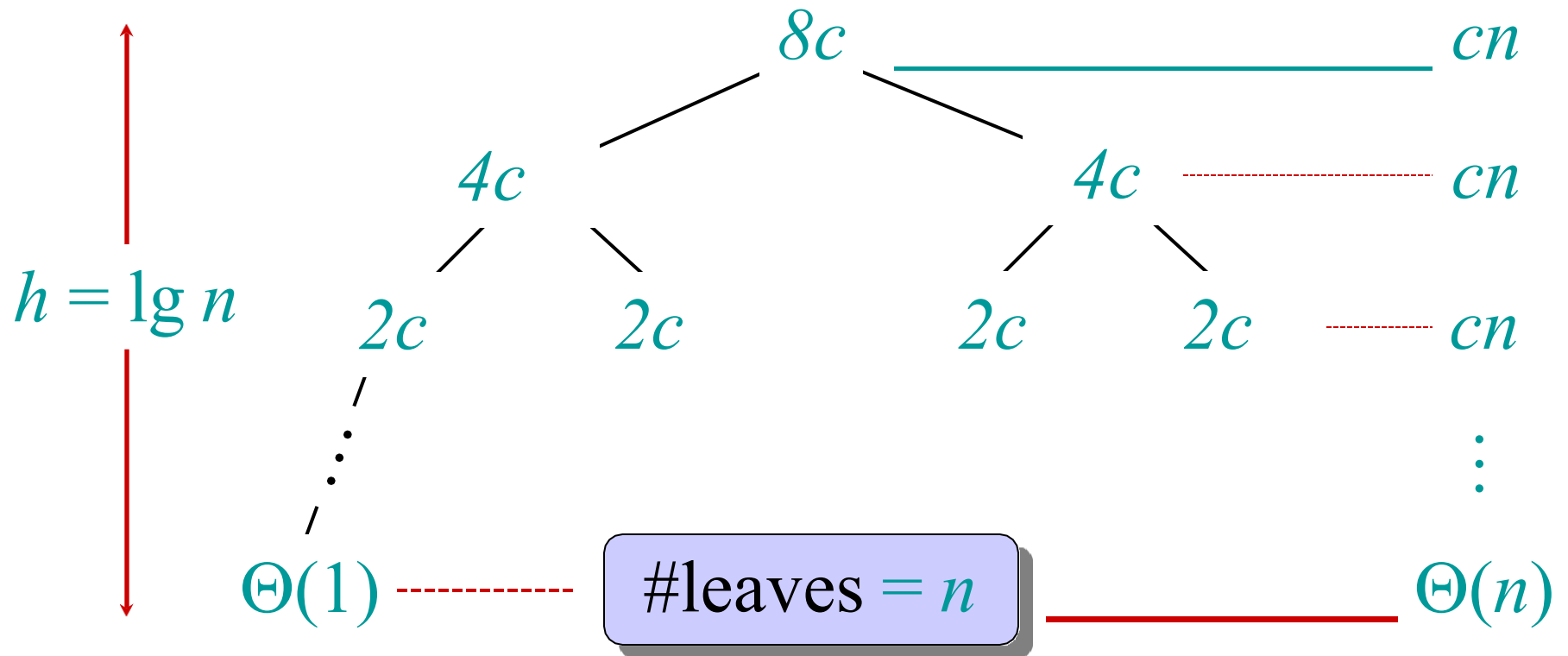


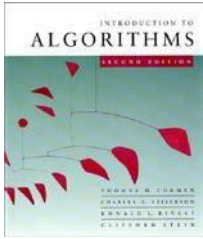


$$n=8$$

Recursion tree

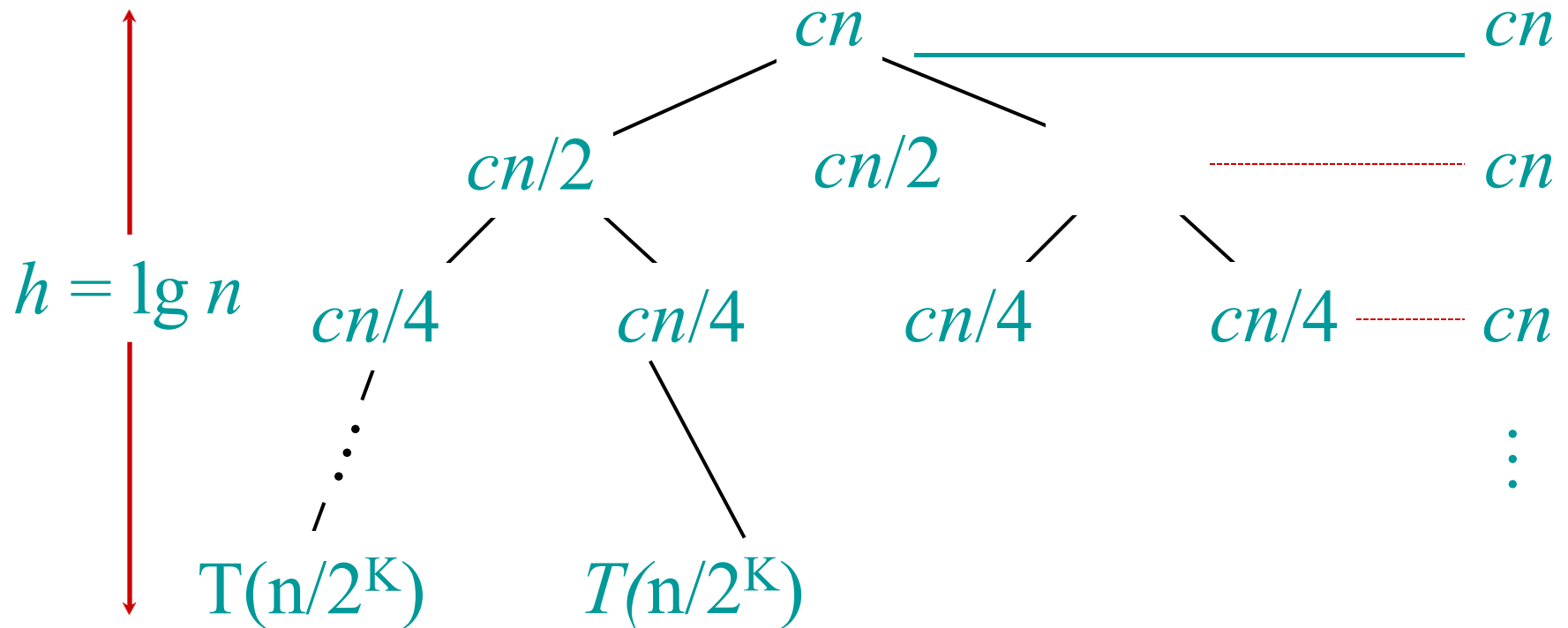
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.





Recursion tree

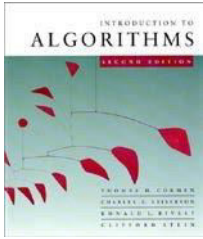
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Number of Iterations

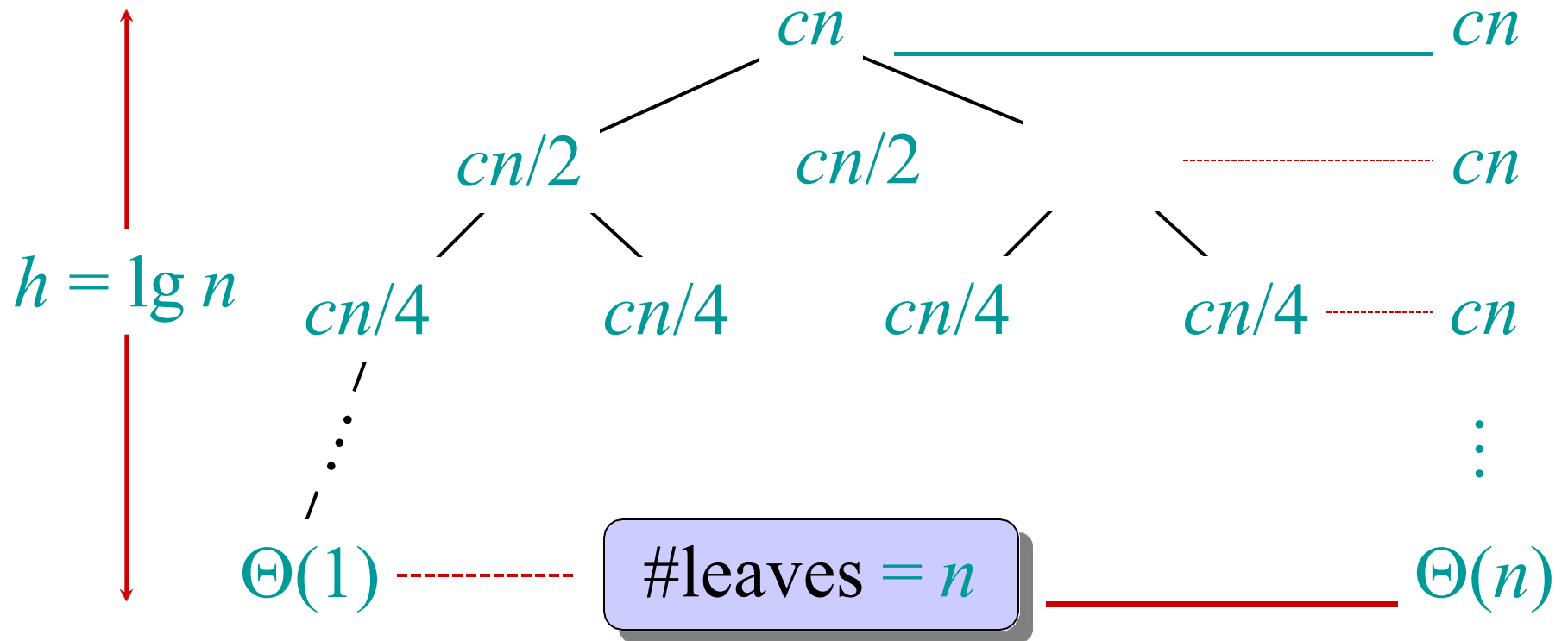
□ $T(n/2^K)$

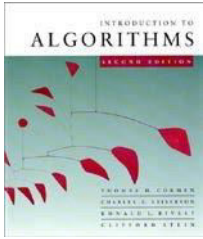
- $n/2^K = 1$
- $n = 2^K$
- $\log(n) = \log(2^K)$
- $\log(n) = k \log 2$
- $\log(n) / \log 2 = k$
- $k = \log(n) / \log 2$
- $k = \lg n$
- Note: Number of iterations = $\lg n$



Recursion tree

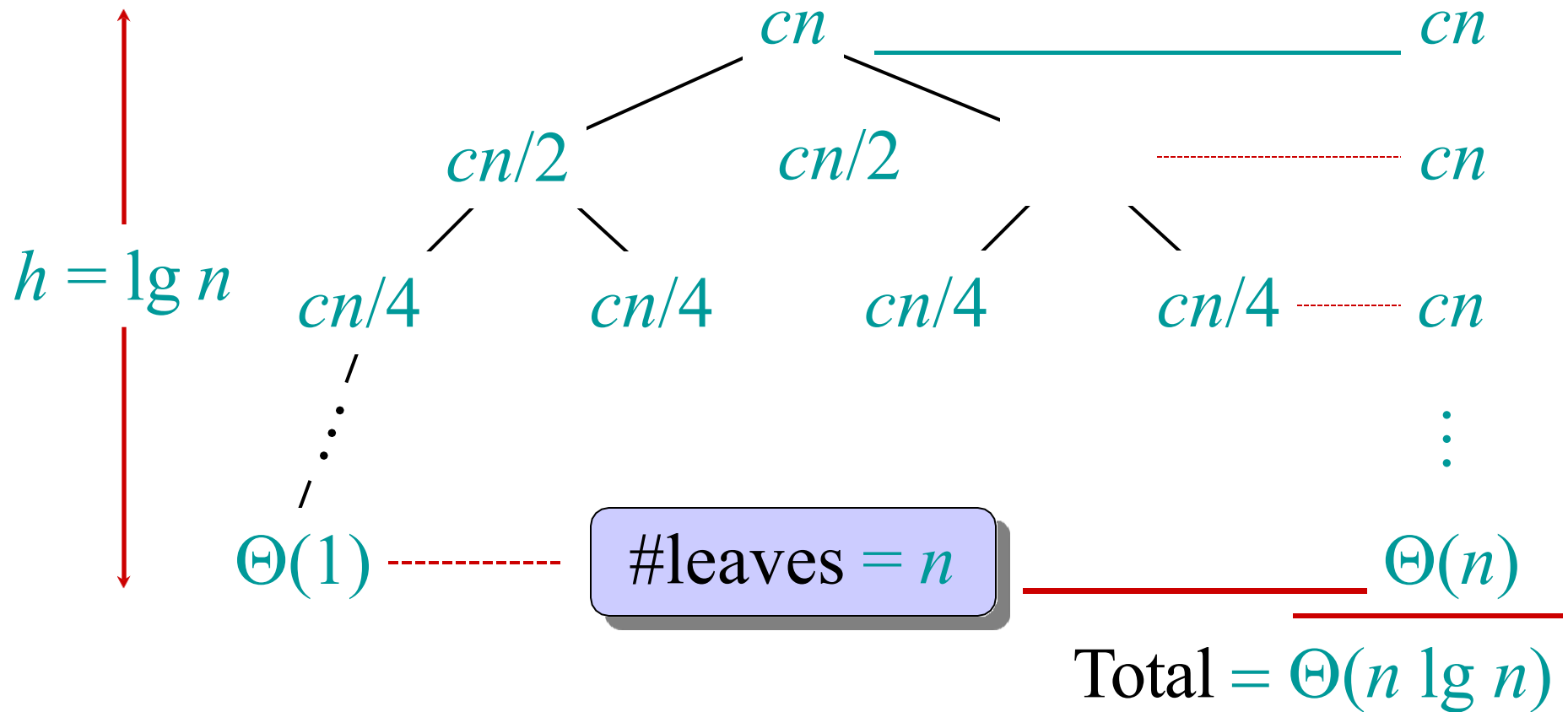
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



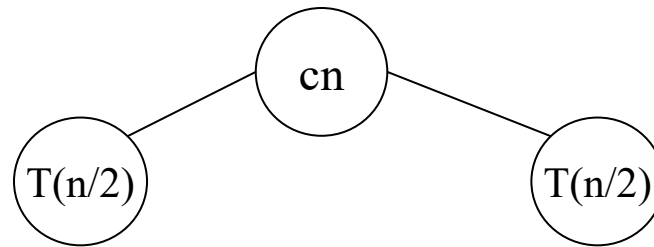


Recursion tree

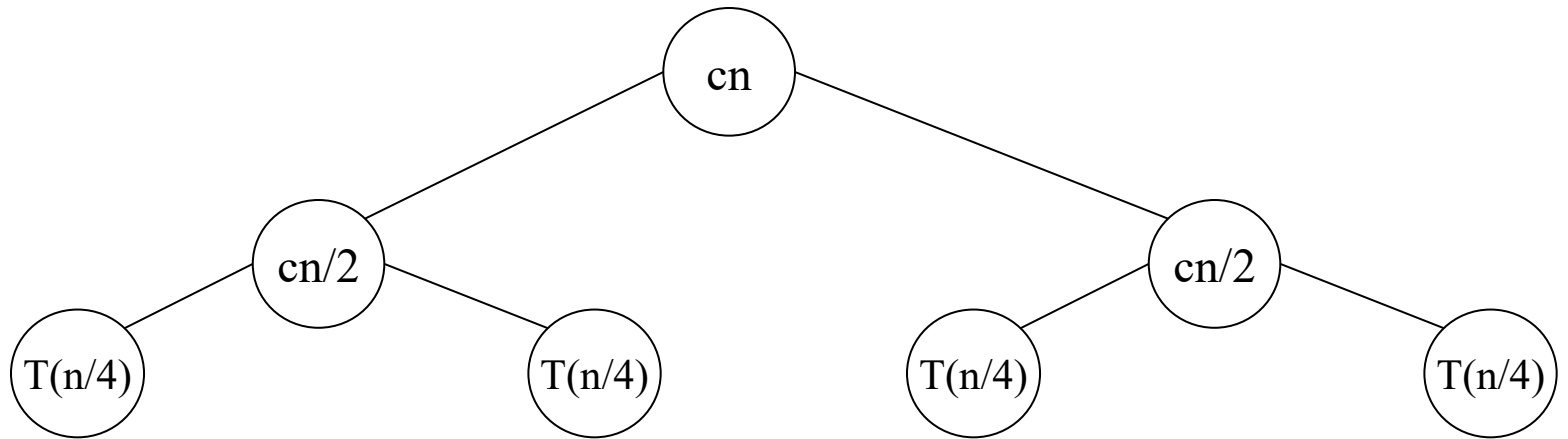
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



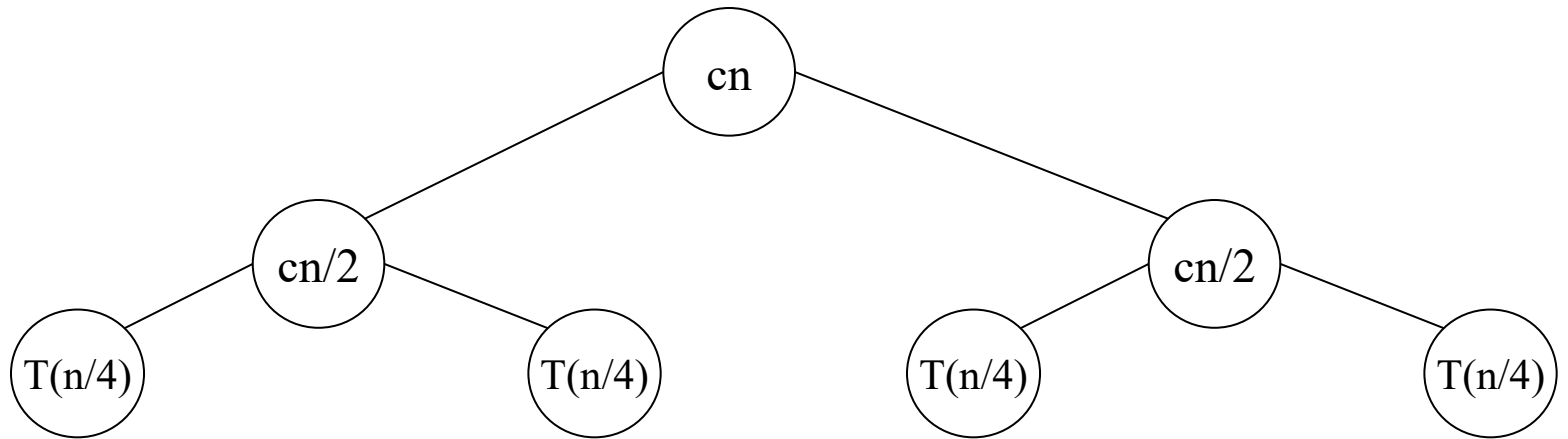
Recursion Tree for Algorithm



Recursion Tree for Algorithm

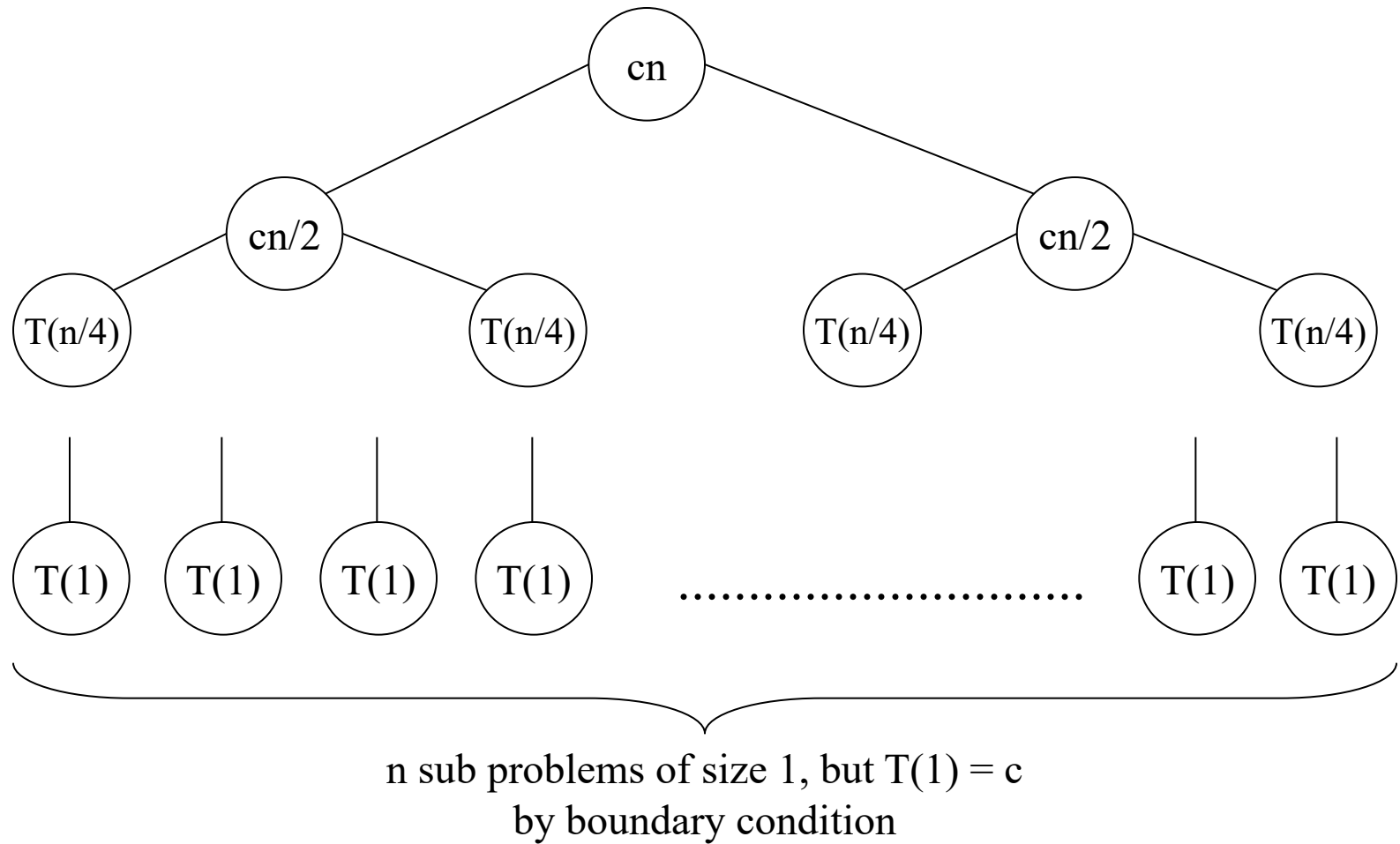


Recursion Tree for Algorithm

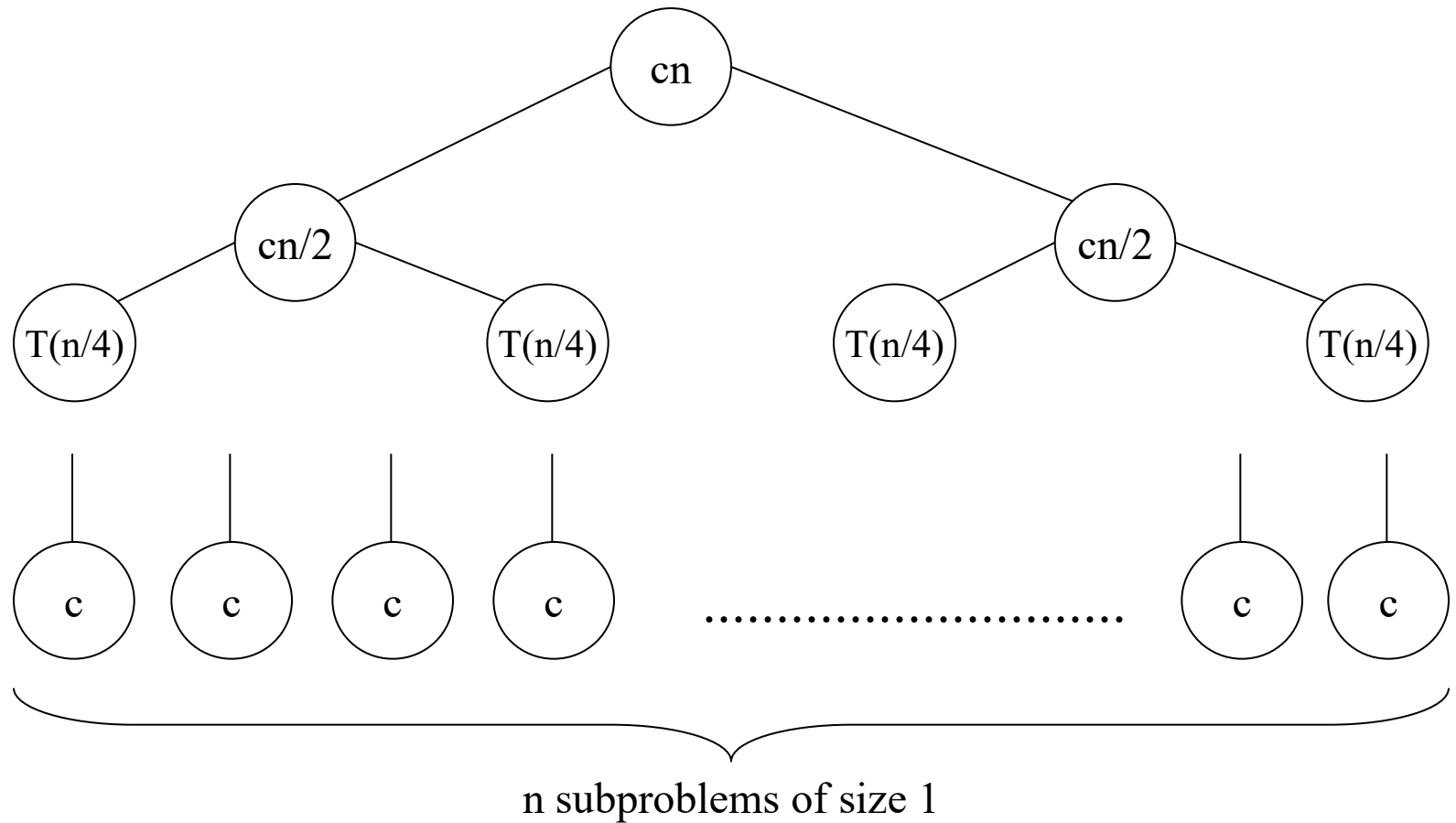


Eventually, the input size (the argument of T) goes to 1, so...

Recursion Tree for Algorithm



Recursion Tree for Algorithm



Recursion Tree for Algorithm

level	nodes/ level	cost/ level
0	$2^0 = 1$	cn
1	$2^1 = 2$	cn
2	$2^2 = 4$	cn
.	.	
.	.	
.	.	
K	$2^k = n$	cn

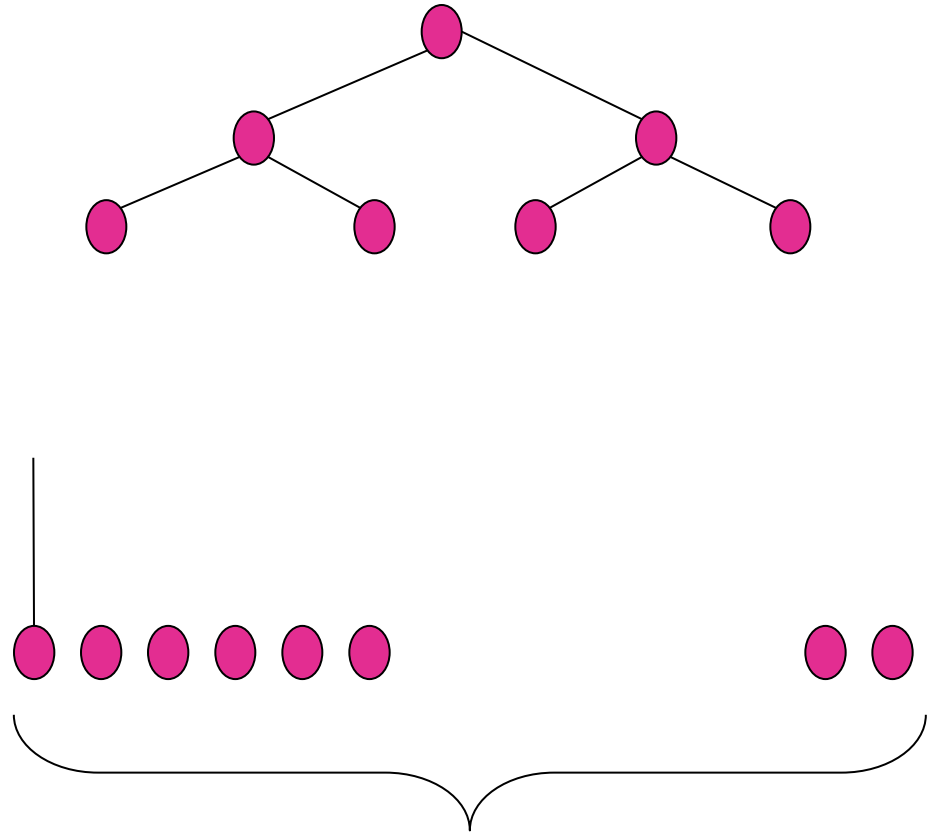
Since $2^k = n$,

$K = \lg(n)$

levels = k = $\lg(n)$

$T(n) = \text{total cost} = (\text{levels})(\text{cost/level})$

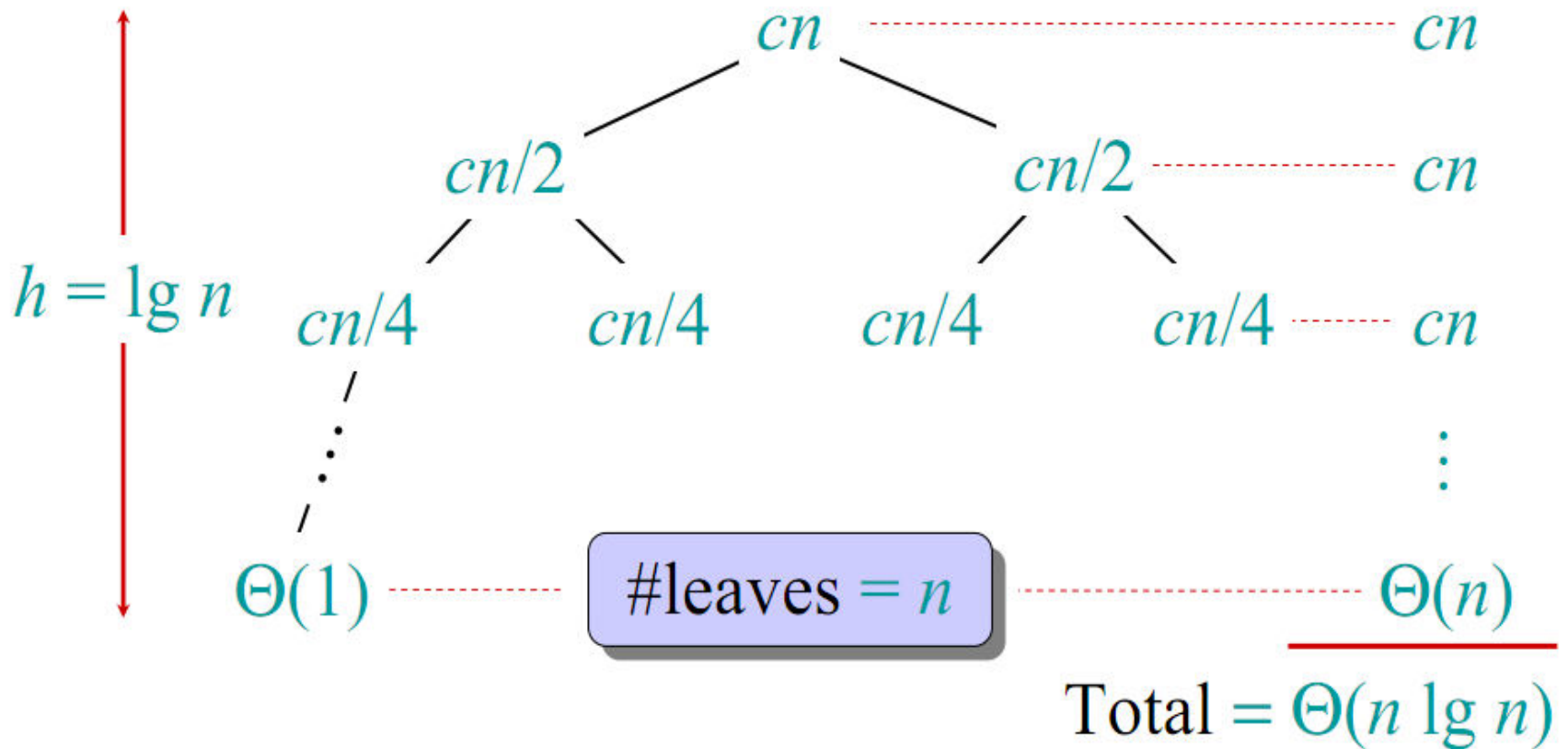
$T(n) = cn [1 + \lg(n)] = O(n \lg(n))$



n nodes at level N-1

Visual Representation of the Recurrence for Merge Sort

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Conclusions

- $\Theta(n \lg n)$ grows more slowly than $\Theta(n^2)$
- Therefore, **merge sort asymptotically beats insertion sort in the worst case.**
- In practice, merge sort beats insertion sort for $n > 30$ or so.
- Go test it out for yourself!

Sorting algorithms

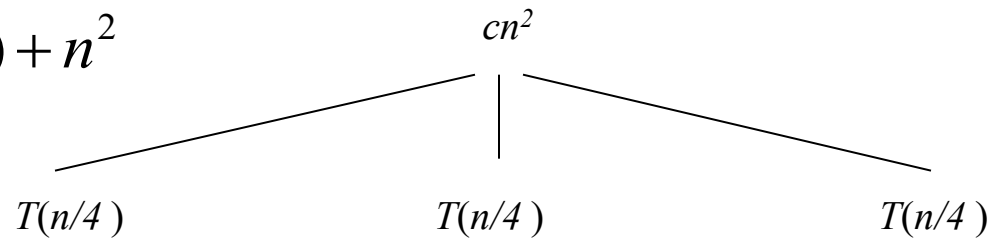
- **Selection and bubble sort have quadratic best/average/worst-case performance**
- **Insertion sort has quadratic average-case and worst-case performance**
- **The faster comparison based algorithm ?
 $O(n \log n)$**
- **Mergesort and Quicksort**

Recursion Tree

$$T(n) = 3T(n/4) + n^2$$

Recursion Tree

$$T(n) = 3T(n/4) + n^2$$

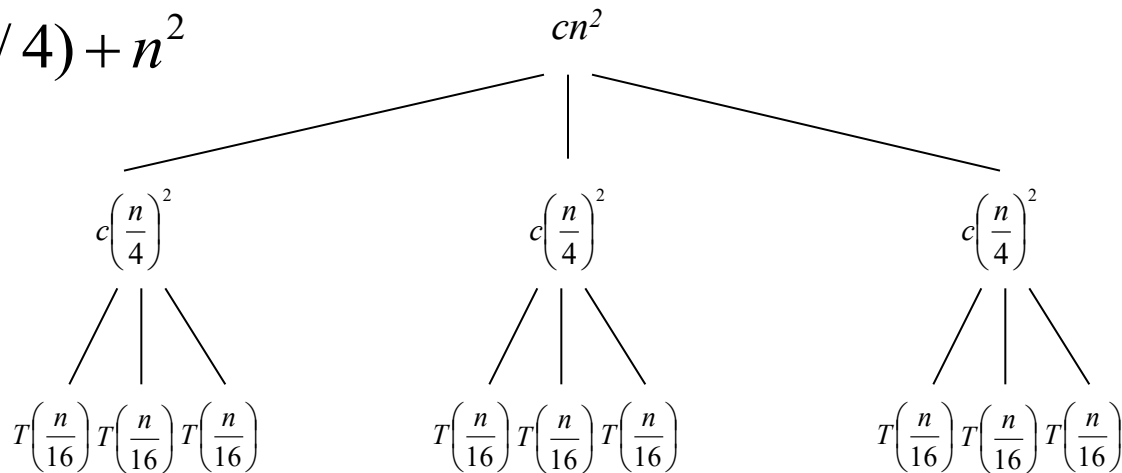


cost

cn^2

Recursion Tree

$$T(n) = 3T(n/4) + n^2$$



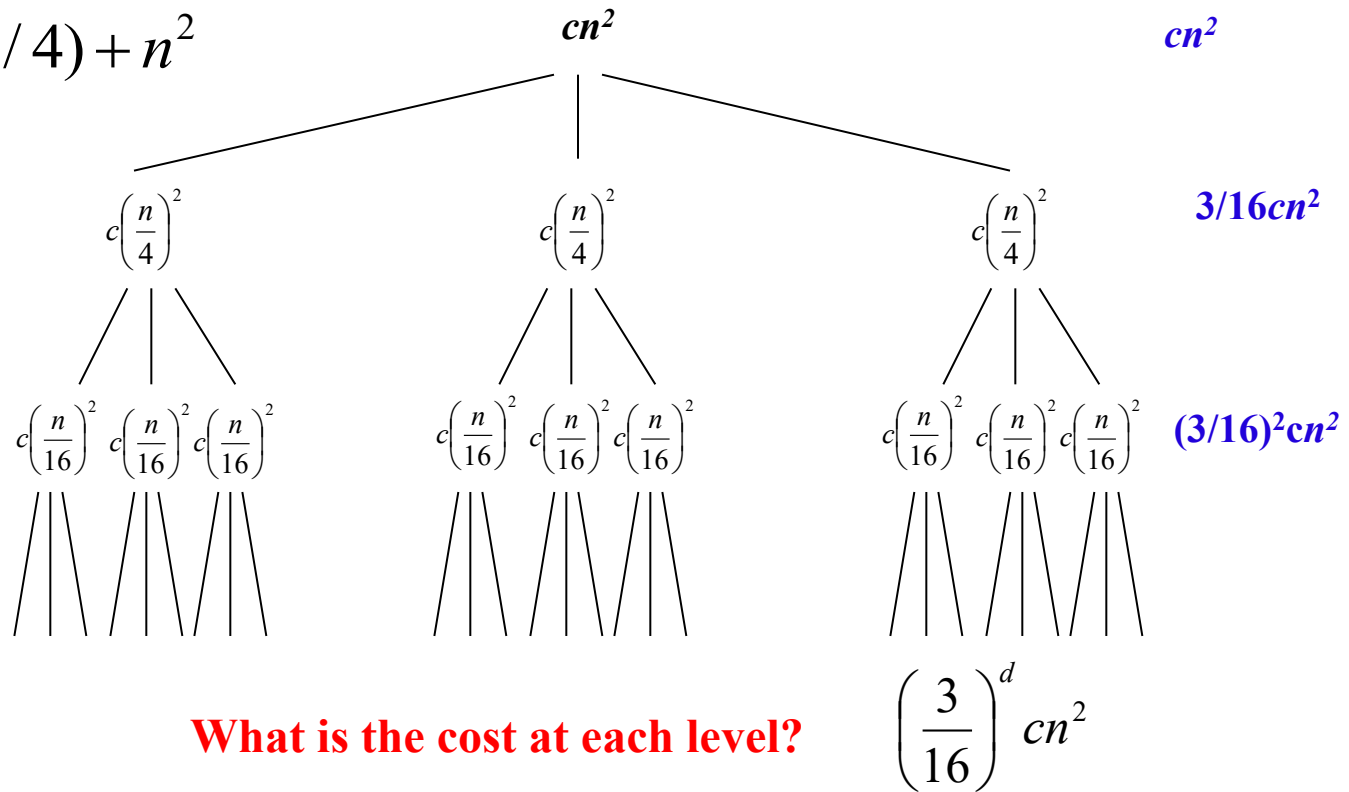
cost

cn^2

$3/16cn^2$

Recursion Tree

$$T(n) = 3T(n/4) + n^2$$



Depth of Tree?

- At each level, the size of the data is divided by 4

$$\frac{n}{4^d} = 1$$

$$\log \left(\frac{n}{4^d} \right) = 0$$

$$\log n - \log 4^d = 0$$

$$d \log 4 = \log n$$

$$d = \log_4 n$$

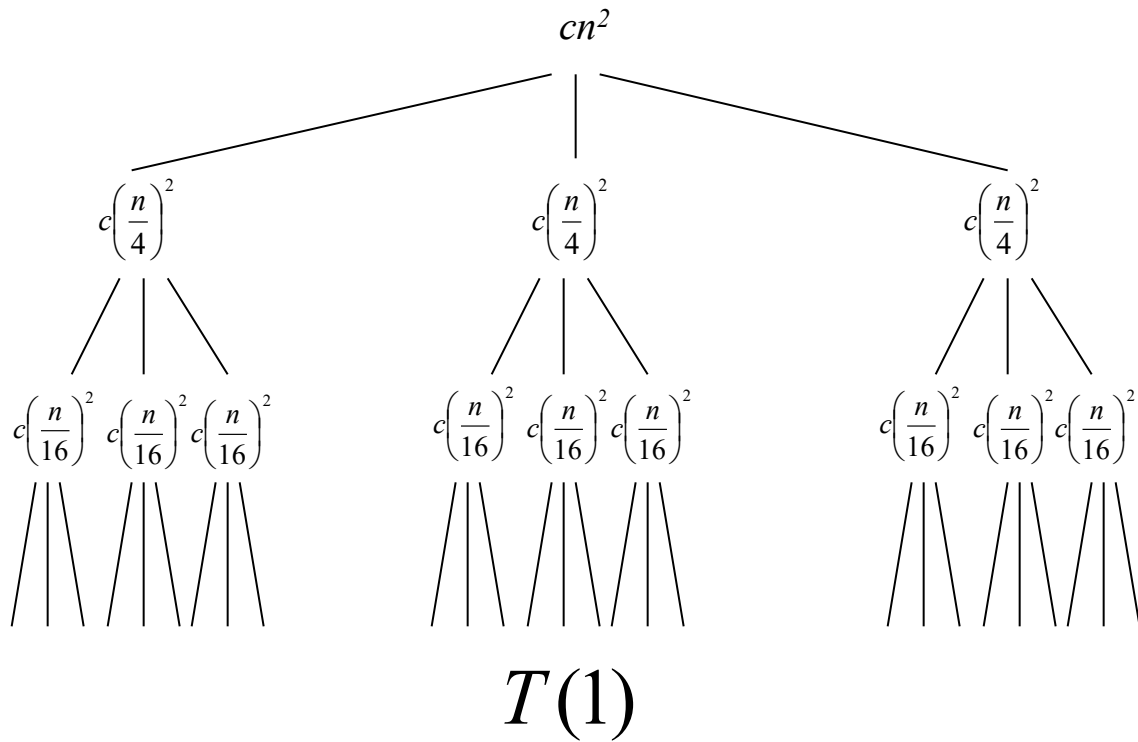
d shows level number

If $n=64$ then depth would be 3, [which is 4 raise to power 3]

Formula for:
log base 4 is
 $\log 64 / \log 4 = 3$



How many Leaves?



How many Leaves?

- How many leaves are there in a complete tree of depth d ?

$$3^d = 3^{\log_4 n}$$

If $n=64$

$d=3$

The leaves would be

$$27=3^3$$

Total Cost

$$T(n) = cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \dots + \left(\frac{3}{16}\right)^{d-1} cn^2 + 3^{\log_4 n}$$

$$= cn^2 \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i + 3^{\log_4 n}$$

$$< cn^2 \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i + 3^{\log_4 n}$$

$$= \frac{1}{1 - (3/16)} cn^2 + 3^{\log_4 n}$$

$$= \frac{16}{13} cn^2 + 3^{\log_4 n} \quad ?$$

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$$

let $x = 3/16$

Total Cost

$$= \frac{16}{13} cn^2 + 3^{\log_4 n}$$



Take Home Task:

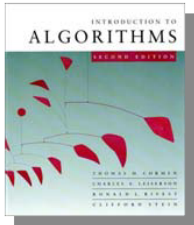
- Check it for large values of n ,
- Draw graph for both of the above functions for large values of n and determine the dominating term

Total Cost

$$\begin{aligned}T(n) &= cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3}) \\&= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\&= \frac{(3/16)^{\log_4 n} - 1}{(3/16) - 1} cn^2 + \Theta(n^{\log_4 3}) \quad (\text{by equation (A.5)}) .\end{aligned}$$

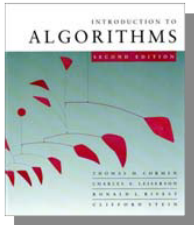
This last formula looks somewhat messy until we realize that we can again take advantage of small amounts of sloppiness and use an infinite decreasing geometric series as an upper bound. Backing up one step and applying equation (A.6), we have

$$\begin{aligned}T(n) &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\&< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\&= \frac{1}{1 - (3/16)} cn^2 + \Theta(n^{\log_4 3}) \\&= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3}) \\&= O(n^2) .\end{aligned}$$



Example of recursion tree

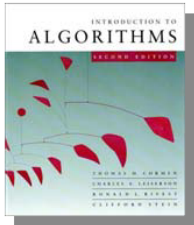
Solve $T(n) = T(n/4) + T(n/2) + n^2$:



Example of recursion tree

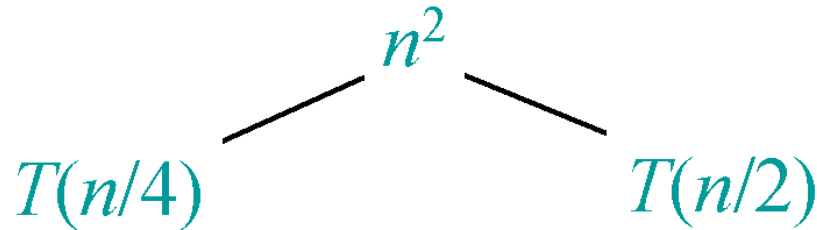
Solve $T(n) = T(n/4) + T(n/2) + n^2$:

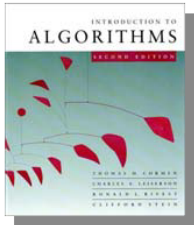
$$T(n)$$



Example of recursion tree

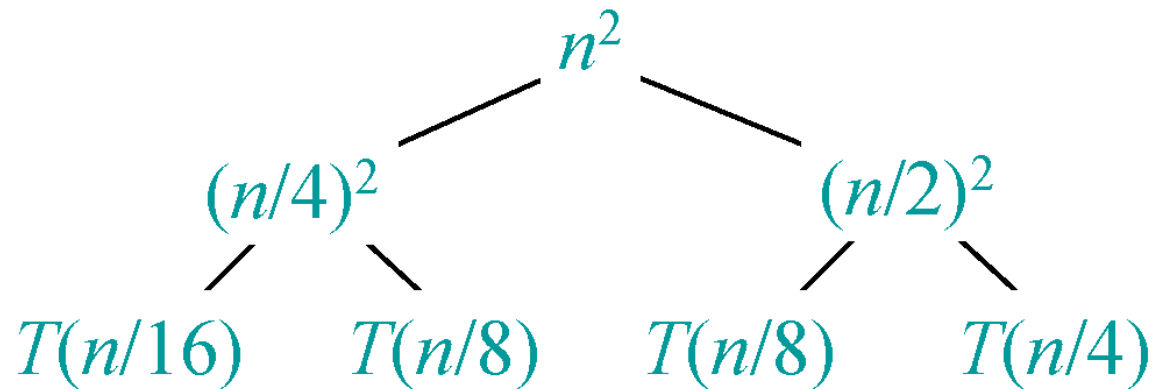
Solve $T(n) = T(n/4) + T(n/2) + n^2$:

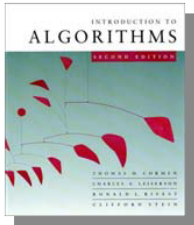




Example of recursion tree

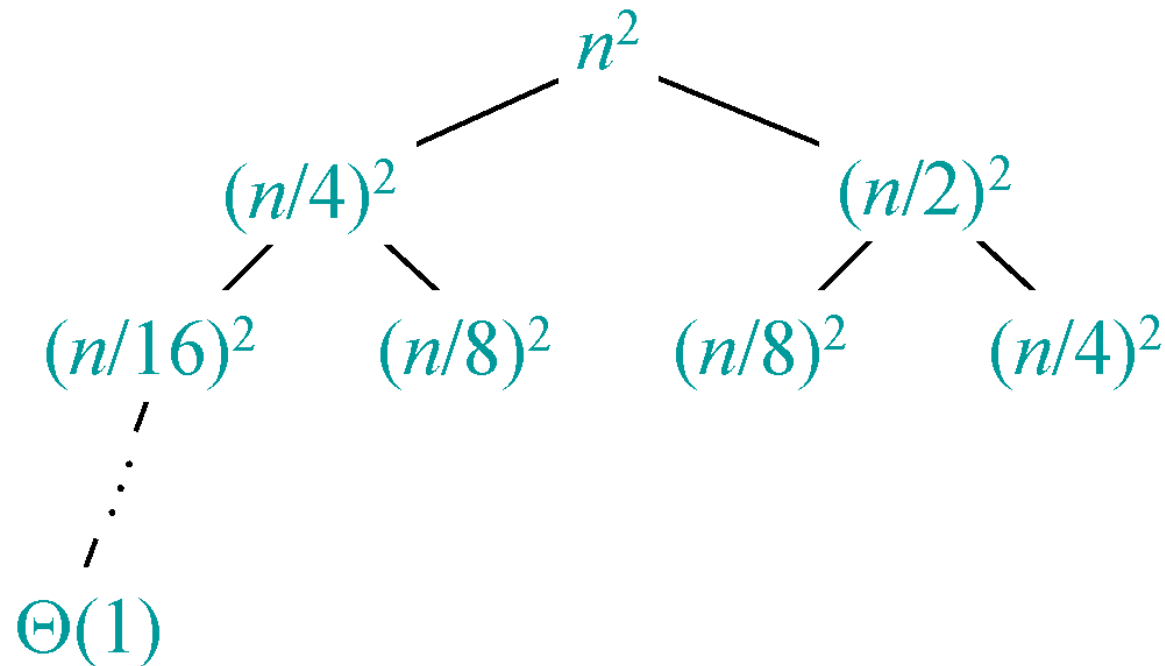
Solve $T(n) = T(n/4) + T(n/2) + n^2$:

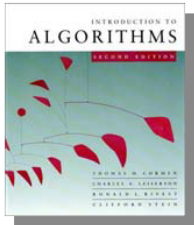




Example of recursion tree

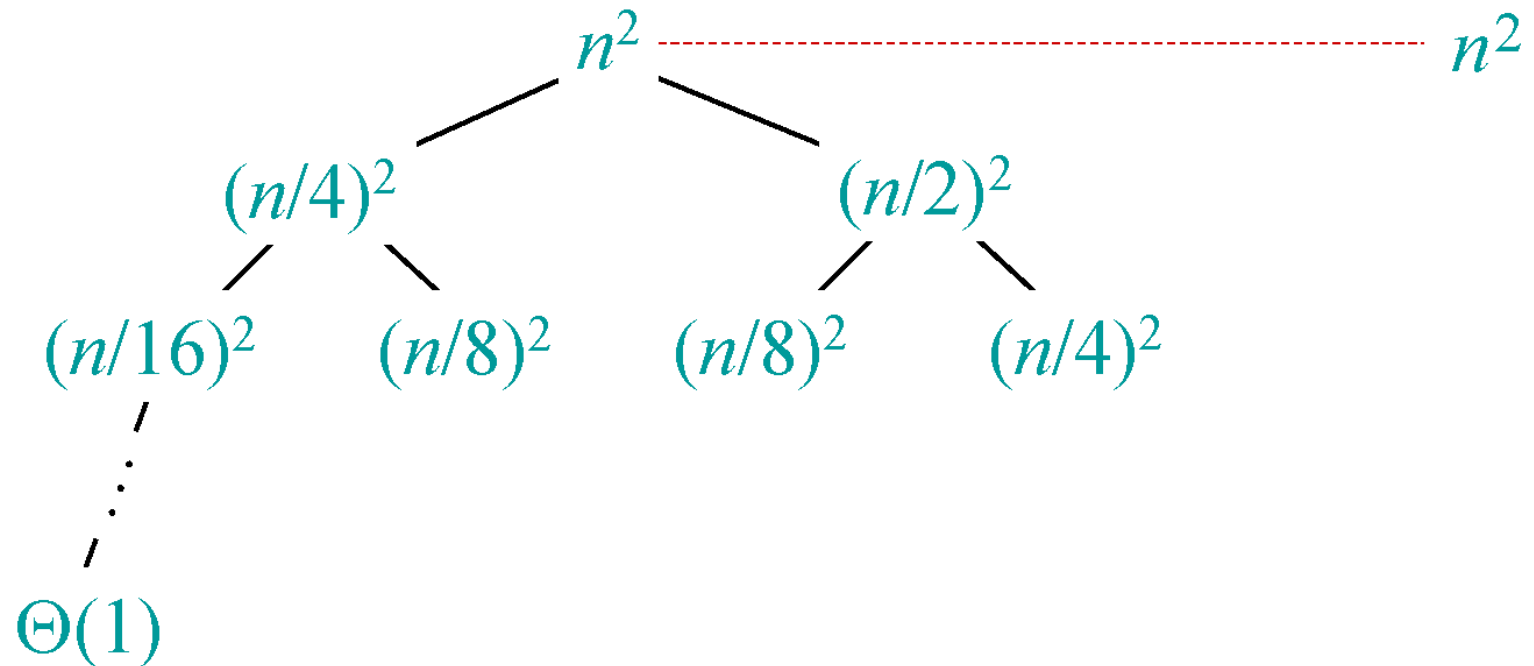
Solve $T(n) = T(n/4) + T(n/2) + n^2$:

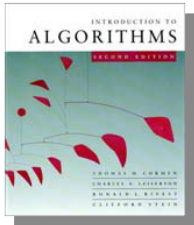




Example of recursion tree

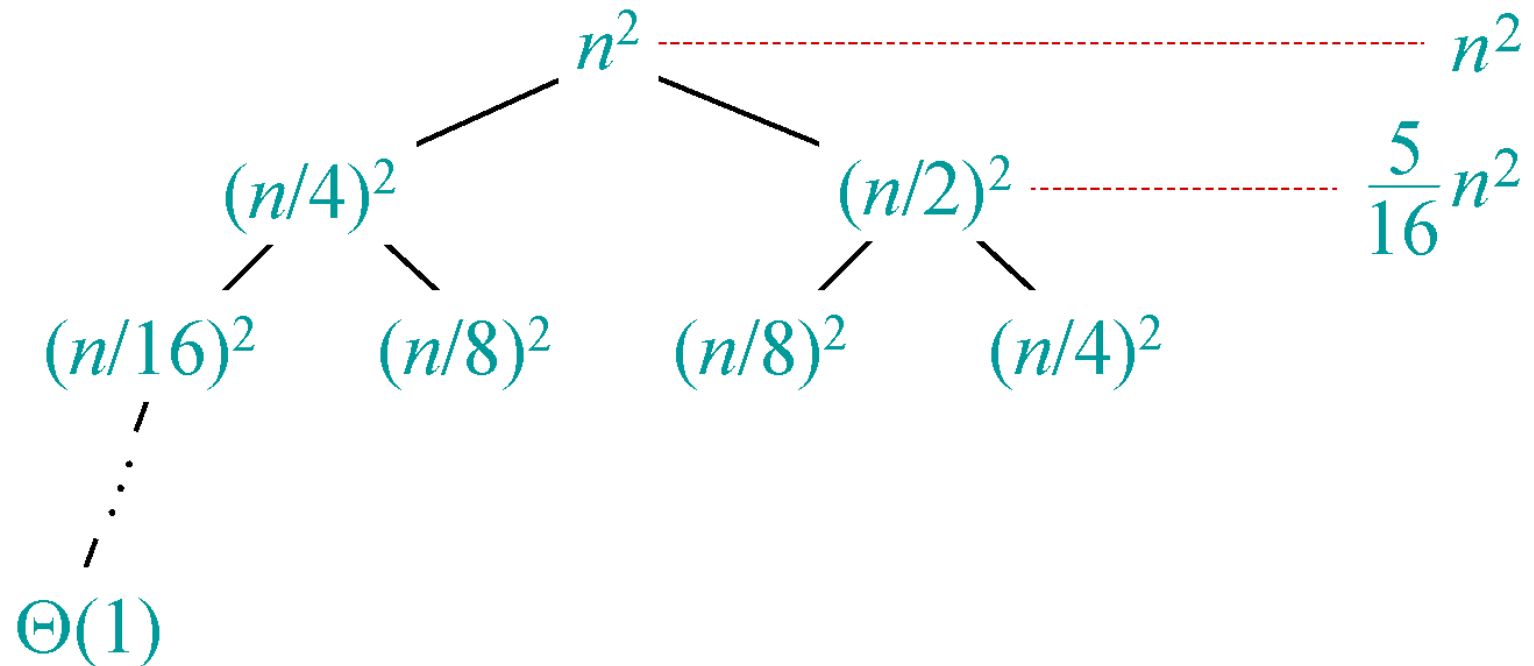
Solve $T(n) = T(n/4) + T(n/2) + n^2$:

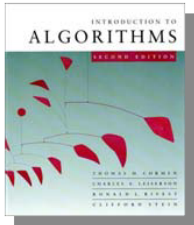




Example of recursion tree

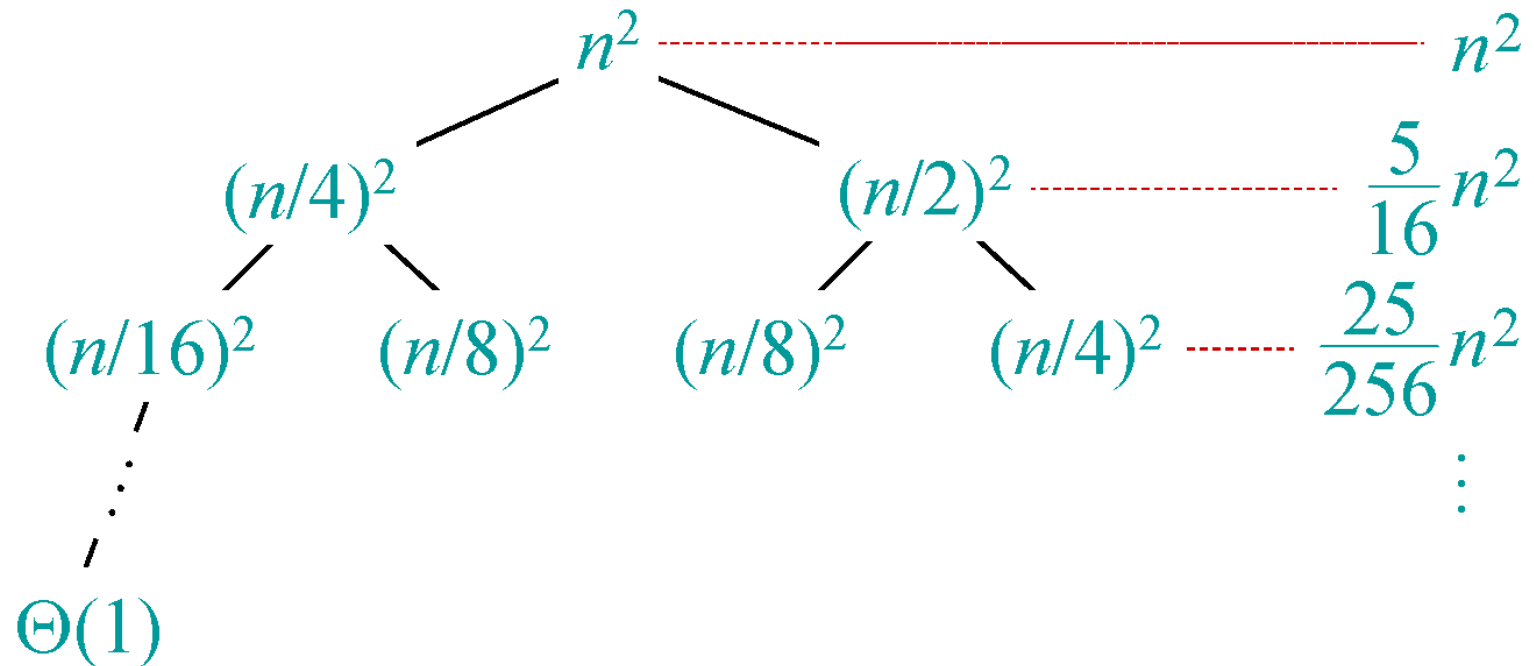
Solve $T(n) = T(n/4) + T(n/2) + n^2$:

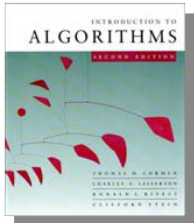




Example of recursion tree

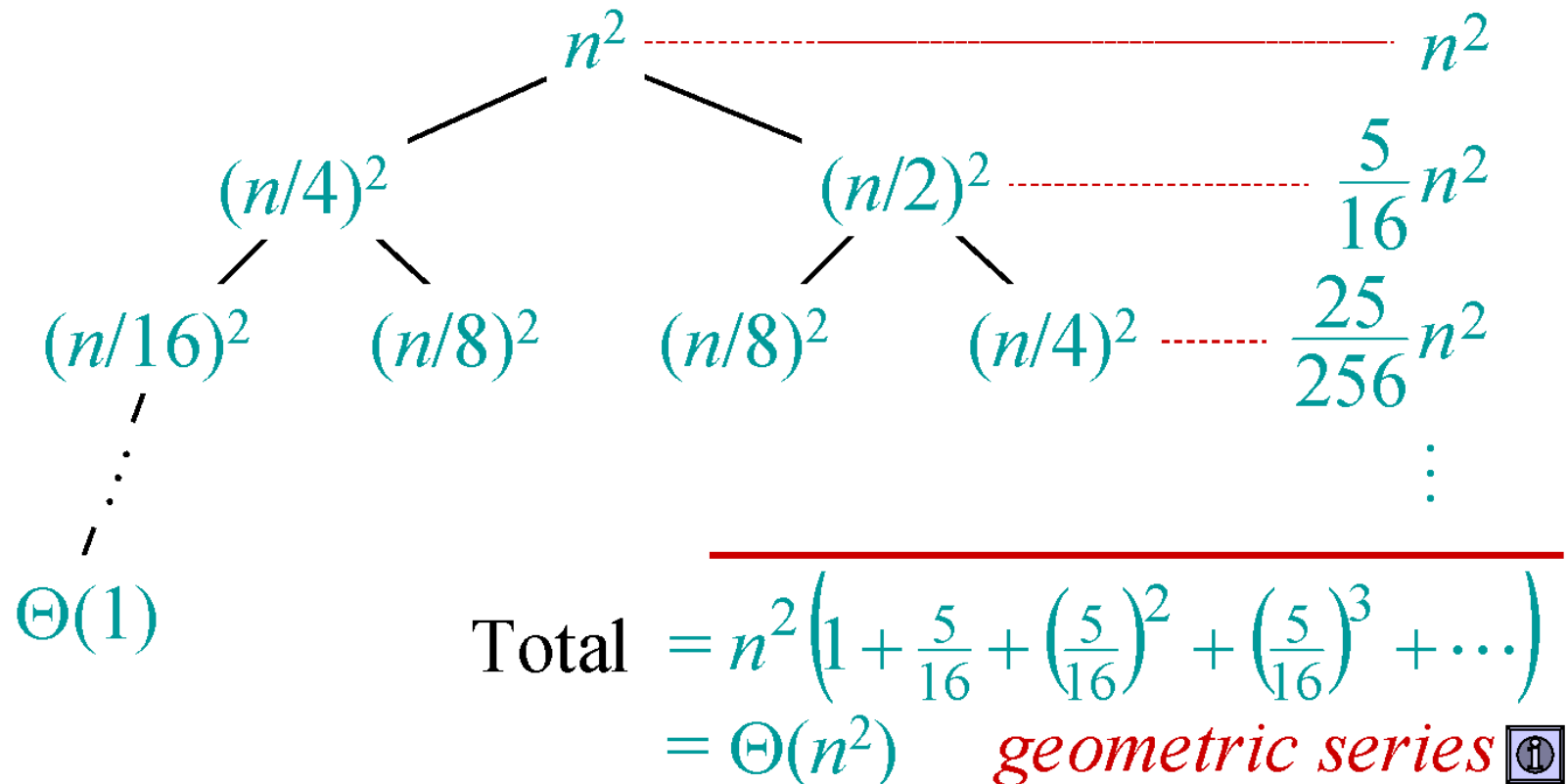
Solve $T(n) = T(n/4) + T(n/2) + n^2$:





Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



Practice

- *Show that*
 $T(n) = 2T(n/2) + n^2$ is $O(n^2)$ by recursion tree and iteration method
- *Show that* $T(n) = T(n/3) + T(2n/3) + n$ is $O(n \log n)$ by recursion tree method

Solution link:

<https://www.cs.cornell.edu/courses/cs3110/2012sp/lectures/lec20-master/lec20.html>

Reference

- **Introduction to Algorithms**
- **4.1, 4.3, 4.4**
 - **Chapter # 4**
 - **Thomas H. Cormen**
 - **3rd Edition**